Research Report (SIT708 9.1P)

Extending the Lost & Found Map App to a Commercial Mobile Application

Why Commercialise

The current Lost & Found app solves a genuine, recurring problem: people lose possessions in public spaces and currently rely on fragmented channels — community social-media groups, physical noticeboards, or word of mouth — that are unstructured and geographically blind. A dedicated, location-aware platform consolidates these reports into a single searchable map, dramatically improving the odds of reuniting items with their owners. A commercial version could serve universities, transit authorities, shopping centres, and event organisers, all of which manage high volumes of lost property and would value a branded, integrated solution. The radius-based search already implemented is the core differentiator: relevance is driven by proximity, which is exactly how people search for lost items in the real world.

How It Would Be Structured

The transition from a single-device prototype to a commercial product requires three architectural shifts.

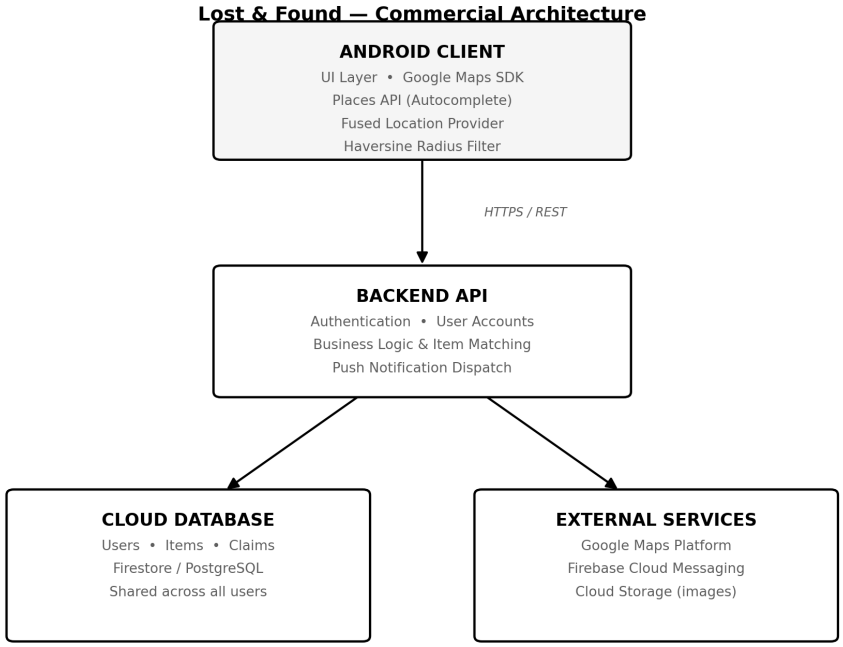
First, a cloud backend. The current Room database is local to one device, so no two users can see each other's posts. A commercial app needs a central server which is a REST API backed by a cloud database such as Firebase Firestore or PostgreSQL, so that all reports are shared across users in real time. The Android client would synchronise with this backend rather than persisting data only on the device.

Second, user accounts and trust. Commercial use demands authentication so that posters are accountable, contact details are protected, and claimants can be verified before an item is handed over, mitigating fraudulent claims. Push notifications would alert users when a matching item appears within their chosen radius.

Third, monetisation and moderation. Revenue could come from organisational subscriptions (for example, a university paying for a branded instance), optional promoted listings, or a small reward-handling fee. Content moderation and reporting tools would be essential to prevent misuse.

The geo-features built in this task — the Google Maps SDK for rendering, the Places API for autocomplete, the Fused Location Provider for current location, and the Haversine radius filter — remain the foundation; commercialisation wraps them in a multi-user, accountable, revenue-generating system.

Architecture Diagram



References

Google. (2026). *Maps SDK for Android overview*. Google for Developers. https://developers.google.com/maps/documentation/android-sdk/overview

Google. (2026). *Set up the Maps SDK for Android*. Google for Developers. https://developers.google.com/maps/documentation/android-sdk/get-api-key

Google. (2026). *Maps SDK for Android — API Reference*. Google for Developers. https://developers.google.com/maps/documentation/android-sdk/reference

Google. (2026). *Places SDK for Android*. Google for Developers. https://developers.google.com/maps/documentation/places/android-sdk/overview

Android Developers. (2026). *Save data in a local database using Room*. https://developer.android.com/training/data-storage/room